

SIMATS ENGINEERING

ASSESSMENT TOOL 1 - Concept Mapping

COURSE NAME: Data Structures

COURSE CODE: CSA0309

Register Number	192525298
Name	M.Dinesh kumar

COURSE OUTCOME COVERED

CO1: Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity. (BL2, BL3)

Assessment Tool Weightage: 35%

Questions:2 EACH QUESTIONS CARRIES: 10 MARKS Total Marks:20 Duration:ONE HOUR

Q1 A smart healthcare system processes large volumes of patient data such as temperature readings, medical history, and diagnostic details. Construct a concept map linking the progression from Data → Data Object → Data Structure → Abstract Data Type (ADT) → Implementation in C. Identify relevant sub-concepts such as structures, pointers, and operations, and establish meaningful relationships among them. Organize the concept map hierarchically and justify how each level contributes to efficient data organization, processing, and improved system performance.

Q2 A smart city traffic management system handles data related to vehicle movement, road networks, and signal control. Construct a concept map connecting Primitive Data Structures → Non-Primitive Data Structures → Linear → Non-Linear Structures. Include examples such as arrays, linked lists, trees, and graphs, and illustrate their applications in traffic management. Explain the relationships between these structures and justify their classification based on functionality and efficiency.

Code of Conduct:

I M.Dinesh kumar(192525298) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

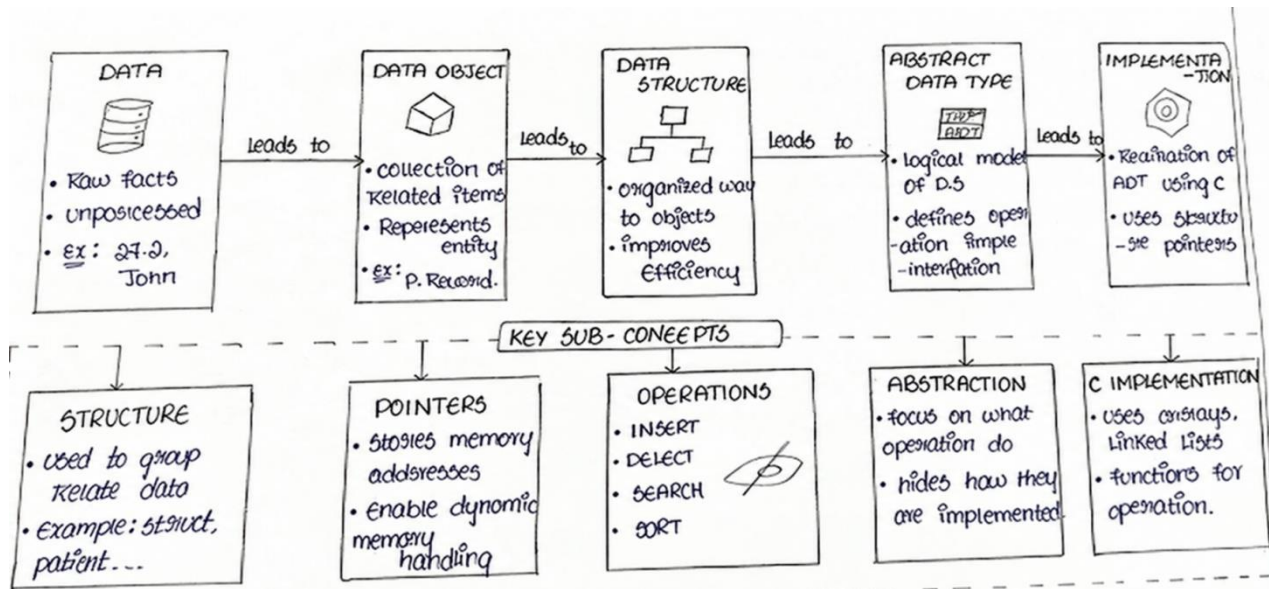
Signature of the student:

M.Dinesh kumar

RUBRICS FOR CALCULATION:

Concept Mapping					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

ANSWER – Q1 (10 Marks)



Explanation of the Concept Map

- Data** – Raw, unprocessed facts in the healthcare system, such as temperature 27.2 / 98.6 °F, a patient name (John) or a diagnostic report.
- Data Object** – A collection of related data items representing one entity, e.g. a Patient record {ID, Name, Temperature, History, Diagnosis}.
- Data Structure** – An organised way of storing data objects in memory (array, structure, linked list) that improves efficiency of access and processing.
- Abstract Data Type (ADT)** – A logical model of the data structure that defines operations (insert, delete, search, sort) and hides how they are implemented.
- Implementation in C** – Realisation of the ADT using structures, pointers, arrays, linked lists and functions written for each operation.

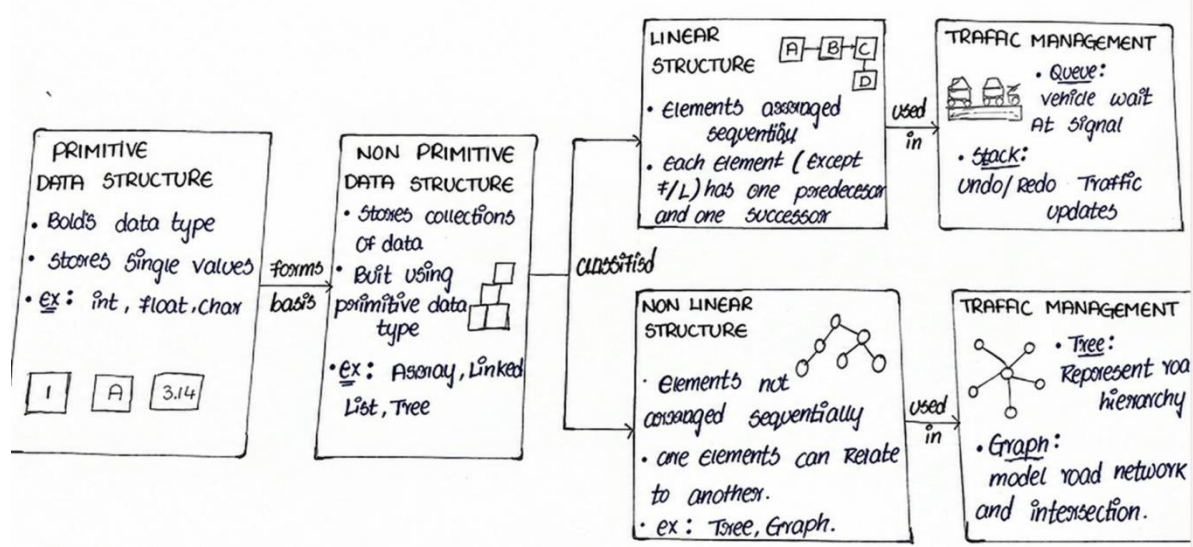
Justification – How Each Level Improves Performance

- Data → Data Object: grouping readings, history and diagnosis into one record avoids scattered and inconsistent data.
- Data Structure: arrays give O(1) access to temperature readings; linked lists allow records to grow without shifting data.
- ADT: standard operations make processing uniform and safe, and the implementation can change without affecting the application.
- C Implementation: pointers and dynamic memory avoid wasted space; modular functions make processing faster and reliable.

Conclusion

The progression Data → Data Object → Data Structure → ADT → Implementation adds organisation at every level. Together they help the smart healthcare system store, search and update large volumes of patient data efficiently, which improves overall system performance.

ANSWER – Q2 (10 Marks)



Explanation of the Concept Map

Primitive Data Structure – Basic data types that store single values, e.g. int (vehicle count), float (speed 3.14), char (signal code R/G). They form the basis of non-primitive structures.

Non-Primitive Data Structure – Stores collections of data and is built using primitive data types, e.g. array, linked list, tree, graph. It is classified into linear and non-linear structures.

Linear Structure – Elements are arranged sequentially; each element (except first and last) has one predecessor and one successor, e.g. array, linked list, stack, queue.

Non-Linear Structure – Elements are not arranged sequentially and one element can relate to many others, e.g. tree and graph.

Applications in Traffic Management

- Queue (FIFO): vehicles waiting at a signal are served in the order they arrive.
- Stack (LIFO): undo/redo of traffic updates, such as reversing a signal-timing change.
- Tree: represents road hierarchy (city → zone → road → junction).
- Graph: models the road network, with intersections as vertices and roads as edges, used for route finding and shortest path.

Relationship and Justification of Classification

- Primitive types form the basis of non-primitive structures, which are classified as linear or non-linear and then applied to traffic problems.
- Functionality: linear structures suit ordered, step-by-step data such as waiting vehicles; non-linear structures suit connected data such as roads and junctions.
- Efficiency: linear structures give simple traversal and $O(1)$ array access; trees give fast searching, and graphs give efficient routing with algorithms such as shortest path.

Conclusion

Classifying data structures as primitive, non-primitive, linear and non-linear helps choose the right structure for each traffic task. This makes handling vehicles, signals and road networks in a smart city faster, organised and efficient.